CityTech TTP Summer Bootcamp

Building an API with Express

2026-06-16

Agenda

1. Review
2. Follow Ups: React keys and Endpoints
3. Complex `fetch` and HTTP Headers
4. API Practice: JSONPlaceholder and SpaceTraders
5. Building an API with Express

Review

What are the most important things you remember from yesterday?

Follow Up: React `key`

When you render a list with `.map()`, React wants a **`key`** on each item:

```
{items.map((item) => <li key={item.id}>{item.name}</li>)}
```

- **Why:** `key` lets React tell items apart between renders - so it updates only what changed and keeps each item's state correct
- **Use a stable, unique id** (e.g. `item.id`) - it stays with the item even if the list reorders
- **Avoid the array index**: if items are added, removed, or reordered, indexes shift and React mismatches them (stale state, subtle bugs)

Follow Up: Terminology - Endpoint

An **endpoint** is a specific **address (URL) your API exposes** - a place a client can send a request to.

- e.g. `https://api.example.com/books/42` is an endpoint
- Pair it with a **method** to describe a full operation: `GET /books/42`
- In Express, every **route** you define is an endpoint
- "Hit the `/books` endpoint" just means *send a request to that path*
- *Endpoint*, *route*, and *path* get used loosely / interchangeably in practice

Complex Node.js `fetch` Calls

A `GET` just reads. A write like `POST` adds a `method`, JSON `headers`, and a stringified `body` (the payload):

```
// GET — read a post
const res = await fetch("https://jsonplaceholder.typicode.com/posts/1");
const post = await res.json();

// POST — create a post
await fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hi", userId: 1 }),
});
```

HTTP Headers

Headers are key-value metadata sent with a request (and its response) - info *about* the request, separate from the body.

```
Content-Type: application/json
Authorization: Bearer <token>
```

- `Content-Type` : the format of your body (e.g. `application/json`)
- `Accept` : the format you want back
- `Authorization` : your credentials, e.g. `Bearer <token>` - **auth**
- `X-API-Key` : an API key, another **auth** style
- `Cookie` : session data sent back to the server - also **auth**

Code Along: JSONPlaceholder

Write a Node script that talks to **JSONPlaceholder** - https://jsonplaceholder.typicode.com

- **GET** `/posts/1` and print its title
- **POST** to `/posts` and log the response
- Try **PATCH** and **DELETE** on `/posts/1`

It accepts writes and returns realistic responses - but doesn't actually persist them.

More API Practice

- Play the **SpaceTraders** game from yesterday - https://spacetraders.io/
- **Tiered learning:** write a **Node.js script** to automate parts of the game - or the whole thing!
- We'll create a class **leaderboard** of players

Express: a Node Server

Express is a minimal web framework for **Node.js** - it lets you build a server that responds to HTTP requests (your own API).

A **route** says: *when a request matches this method + path, run this handler.*

- **Method + path**: e.g. `GET /add/:a/:b`, `GET /multiply/:a/:b`, `GET /square/:n` (`:a`, `:b`, `:n` are **URL parameters**)
- **Handler**: a function `(req, res) => { … }`:
  - `req` - the incoming request (params, query, body, headers)
  - `res` - how you reply (a status code + data, usually `res.json(...)`)
- Express checks routes **top to bottom** and runs the **first match**

Express: Example

```
import express from "express";
const app = express();

app.get("/", (req, res) => {
  res.send("Hello, world!");
});

app.listen(3000);
```

Visit `http://localhost:3000/` → **"Hello, world!"** - a quick smoke test that your server runs.

Express: Setting Up and Running

```
yarn init -y              # set up package.json - only once per project
yarn add express          # install Express into your project
node server.js            # start your server
```

- Put your code in a file (e.g. `server.js`), then run it with **`node`**
- It keeps running - leave it up; visit `http://localhost:3000` to hit it
- **`import express`** needs `"type": "module"` in `package.json` (or a `.mjs` file)
- **Ctrl-C** stops it; restart to pick up code changes - or use **`nodemon`** to auto-restart

Express Is Just a Running Program

An Express server is a **long-running Node program** - you start it and it keeps running, waiting for requests.

- It's **no different from a Node.js script** - just one that doesn't exit
- Anything it stores lives in **RAM** (plain variables)
- So when the server **restarts, that data is gone** - we can add a database later

Express: a POST Route

```
app.use(express.json()); // parse JSON request bodies
let memory = 0;          // the calculator's stored value

app.post("/memory", (req, res) => {
  memory = req.body.value;          // store a number (like "MS")
  res.status(201).json({ memory });
});
```

- `req.body` holds the posted JSON - needs `express.json()`
- Reply **`201 Created`** with the stored value

Express: One URL Parameter

```
app.get("/square/:n", (req, res) => {
  const { n } = req.params;
  res.json({ n, result: Number(n) ** 2 });
});
```

- `:n` in the path → `req.params.n`
- `GET /square/7` → `req.params.n` is `"7"` (a string), so `Number(n)`

Express: a PATCH Route

```
app.patch("/memory", (req, res) => {
  memory += req.body.add;          // add to the stored value ("M+")
  res.json({ memory });
});
```

- **PATCH** updates *part* of a resource (**PUT** replaces the whole thing)
- Reads the amount to add from the JSON body (`req.body`)

Express: a DELETE Route

```
app.delete("/memory", (req, res) => {
  memory = 0;                          // clear the stored value ("MC")
  res.status(204).end();               // No Content
});
```

- Clears the calculator's memory back to zero
- `204 No Content` is common after a successful delete

Express: Two URL Parameters

```
app.get("/add/:a/:b", (req, res) => {
  const { a, b } = req.params;
  res.json({ result: Number(a) + Number(b) });
});
```

- Each `:name` segment becomes a key on `req.params`
- `GET /add/3/4` → `{ result: 7 }` (params arrive as strings, so `Number(...)`)